ORGNZ-99: Best Practice Summary & DQL Reference

> **Series:** ORGNZ — Organize Data: Buckets, Segments, Security | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 07/24/2026

Overview

This notebook consolidates every actionable best practice from the ORGNZ series (notebooks 01-10) into a single definitive reference. Each section provides the exact setting, value, or action to take — and a **Validation Queries** subsection with executable DQL to verify your configuration against a live Grail environment.

Table of Contents

> Each section ends with a **Validation Queries** block — executable DQL to verify your configuration against a live Grail environment.

---

1. Bucket Design

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Keep per-bucket ingest under 1 TB/day	Target: <1 TB/day per bucket; split at >2 TB/day (Dynatrace guidance)	Critical	Performance
2	One data type per bucket	Each bucket stores exactly one table type (logs, metrics, spans, events, bizevents, security_events)	Critical	Architecture
3	Use consistent naming convention	Format: `<provider>_<type>_<retention>` or `<team>_<type>_<retention>` (e.g., `aws_logs_35d`, `team_platform_logs`)	Critical	Naming
4	Bucket names: lowercase, letter-first, underscores/hyphens only	Regex: `^[a-z][a-z0-9_-]*$`; never use `default_*` prefix	Critical	Naming
5	Plan bucket names before creation	Bucket names are immutable after creation; display names can be changed	Critical	Planning
6	Stay within the 80-bucket environment limit	Budget buckets across teams; request increase only if justified	Recommended	Planning
7	Create dedicated buckets per cost center	One bucket per business unit/LOB for direct GiB/day cost attribution	Recommended	Cost
8	Create separate buckets for compliance data	Dedicated bucket (e.g., `compliance_audit_logs`) with extended retention for regulatory requirements	Recommended	Compliance
9	Create short-retention buckets for debug logs	Bucket: `debug_logs_7d` with 7-day retention for high-volume verbose logs	Recommended	Cost
10	Split high-volume buckets to maintain queryability	If bucket exceeds 1 TB/day, split by team, environment, or application	Recommended	Performance

Validation Queries — Bucket Design

List all buckets with data type, retention, and estimated storage:

```
// Inventory of all Grail buckets — verify data types, retention, and naming convention
fetch dt.system.buckets
| fields name, display_name, dt.system.table, retention_days, estimated_uncompressed_bytes
| sort dt.system.table asc, name asc
```

List all built-in buckets (use to compare your environment to the expected defaults):

```
// Discover all built-in Grail buckets — compare against expected defaults
fetch dt.system.buckets
| filter startsWith(name, "default_") or startsWith(name, "dt_")
| fields name, display_name, dt.system.table, retention_days
| sort dt.system.table asc, name asc
```

2. Retention Strategy

#	Best Practice	Recommended Setting/Value	Priority	Category
11	Set debug/development log retention to 3-7 days	Bucket retention: 7 days maximum for DEBUG-level logs	Critical	Cost
12	Set standard operational log retention to 35 days	Bucket retention: 35 days for application and infrastructure logs	Recommended	Operations
13	Set performance analysis retention to 60-90 days	Bucket retention: 90 days for metrics and quarterly trend analysis	Recommended	Operations
14	Set audit/compliance log retention to 365-3657 days	Bucket retention: match regulatory requirement (SOX: 7 years = 2,555 days; HIPAA: 6 years = 2,190 days)	Critical	Compliance
15	Set span retention to 10-35 days	Bucket retention: 10 days for default APM; 35 days if long-running incident correlation needed	Recommended	Operations
16	Set security event retention to 90-365 days	Bucket retention: 365 days for security investigation timelines	Recommended	Security
17	Never lower retention without understanding data loss	Reducing retention immediately deletes data older than the new setting; this is irreversible	Critical	Operations

Validation Queries — Retention Strategy

Compare retention settings across all bucket types:

```
// Compare bucket retention configurations — verify each tier matches its compliance
requirement
fetch dt.system.buckets
| fields name, dt.system.table, retention_days
| sort retention_days desc
```

3. Data Routing

#	Best Practice	Recommended Setting/Value	Priority	Category
18	Route data to custom buckets via OpenPipeline	Configure OpenPipeline routing rules with conditions on `loglevel`, `host.group`, `service.name`, `log.source`	Critical	Architecture
19	Always define a default route	Set `default: "default_logs"` (or appropriate default bucket) as the fallback in every OpenPipeline routing configuration	Critical	Reliability
20	Route DEBUG logs to short-retention buckets	OpenPipeline condition: `loglevel == 'DEBUG'` → destination: `debug_logs_7d`	Recommended	Cost
21	Route audit logs to compliance buckets	OpenPipeline condition: `log.source contains 'audit'` → destination: `audit_logs_365d`	Recommended	Compliance
22	Data cannot be moved between buckets after ingestion	Route new data via OpenPipeline; old data stays in the original bucket until retention expires	Critical	Architecture

Validation Queries — Data Routing

Verify routing rules are directing data to the expected buckets:

```
// Audit log volume by bucket — verify routing rules are directing data correctly
fetch logs, from:-1h
| summarize recordCount = count(), by:{dt.system.bucket}
| sort recordCount desc
```

```
// Track log ingest trends by bucket over time — identify routing anomalies
fetch logs, from:-1h
| summarize recordCount = count(), by:{time_bucket = bin(timestamp, 1h), dt.system.bucket}
| sort time_bucket desc
```

4. IAM and Access Control

#	Best Practice	Recommended Setting/Value	Priority	Category
23	Always include `storage:buckets:read` in every data access policy	Without bucket read, users cannot query any data; always pair with table permissions	Critical	Security
24	Assign policies to groups, never to individual users	Create IAM groups per team/role; assign all policies at the group level	Critical	Governance
25	Use the two-step policy pattern	Step 1: `ALLOW storage:buckets:read WHERE ...`; Step 2: `ALLOW storage:logs:read` (or other table types)	Critical	Security
26	Use STARTSWITH for team bucket prefixes	Policy: `ALLOW storage:buckets:read WHERE storage:bucket-name STARTSWITH "team_platform_"` — automatically includes future buckets with matching prefix	Recommended	Scalability
27	Use MATCH operator for array fields	When `dt.security_context` holds an array, use `MATCH` (not `=`, `STARTSWITH`, or `IN` — those always return false for arrays)	Critical	Security
28	Start with least privilege	Grant minimum required access; expand only when justified	Critical	Security
29	Use policy boundaries for reusable conditions	Define regional/environmental restrictions once as a boundary; apply to multiple policies	Recommended	Governance
30	Limit policy boundaries to 10 restrictions per boundary	Maximum 10 conditions per boundary; create additional boundaries if more are needed	Recommended	Architecture
31	Use field-level DENY for sensitive data	Policy: `DENY storage:logs:read:user.email, storage:logs:read:user.ip_address` to mask PII fields	Recommended	Security
32	Test policies with sample users before deployment	Create a test user in the target group and verify query results before rolling out to the team	Critical	Operations
33	Document every IAM policy	Include name, description, tags, target group, and purpose in a policy registry	Recommended	Governance

Validation Queries — IAM and Access Control

Verify bucket access and data distribution across teams:

```
// Verify bucket access — distribution of accessible log data by bucket
fetch logs, from:-1h
| summarize count = count(), by:{dt.system.bucket}
| sort count desc
```

```
// Check namespace-based access — verify namespace-scoped policies return expected data
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name)
| summarize count = count(), by:{k8s.namespace.name}
| sort count desc
```

```
// Check host group-based access — verify host-group-scoped policies return expected data
fetch logs, from:-1h
| filter isNotNull(dt.host_group.id)
| summarize count = count(), by:{dt.host_group.id}
| sort count desc
```

5. Security Context

#	Best Practice	Recommended Setting/Value	Priority	Category
34	Use `dt.security_context` for record-level access control	Set via OpenPipeline Security Context processor or OneAgent host properties	Critical	Security
35	Use hierarchical encoding in security context values	Format: `<org>/<department>/<team>` (e.g., `acme/engineering/platform`); enables department-level access with `MATCH ('acme/engineering/*')`	Critical	Scalability
36	Use type prefixes for security context values	Prefix with `team:`, `lob:`, `env:`, `cloud:` (e.g., `team:checkout`, `lob:finance`, `env:production`)	Recommended	Naming
37	Plan security context hierarchy before implementation	Context values are assigned to records at ingest time; changing schema requires re-ingesting data	Critical	Planning
38	Use security context instead of additional buckets when at 80-bucket limit	Security context enables multi-team isolation within shared buckets without consuming bucket quota	Recommended	Scalability

#	Best Practice	Recommended Setting/Value	Priority	Category
39	Set security context on entities via Grail Security Context settings	Settings → Topology model → Grail Security Context; entities inherit context to related records	Recommended	Security
40	Always use MATCH for array-valued security context in IAM policies	`ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ("team-a*")`	Critical	Security

Validation Queries — Security Context

Verify `dt.security_context` assignment, distribution, and hierarchical patterns:

```
// View logs with security context — confirm dt.security_context is being set
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| fields timestamp, content, dt.security_context
| limit 20
```

```
// Summarize log volume by security context — verify naming conventions and distribution
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize recordCount = count(), by:{dt.security_context}
| sort recordCount desc
| limit 20
```

```
// Filter logs by specific security context — test access control boundary for a team
fetch logs, from:-1h
| filter dt.security_context == "team:platform"
| limit 50
```

```
// Check hierarchical security context — verify parent-level wildcard access works
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| filter startsWith(dt.security_context, "finance/")
| summarize count = count(), by:{dt.security_context}
| sort count desc
```

```
// Audit security context coverage — percentage of logs with dt.security_context assigned
fetch logs, from:-1h
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    withoutContext = countIf(isNull(dt.security_context))
| fieldsAdd coverage = round(toDouble(withContext) / toDouble(total) * 100, decimals: 2)
```

6. Segment Design

#	Best Practice	Recommended Setting/Value	Priority	Category
41	Use segments for dynamic data views, not access control	Segments filter at query time; they do not enforce security boundaries — IAM policies do	Critical	Architecture
42	Stay within the 20-include limit per segment	Maximum 20 include blocks per segment; plan rule complexity accordingly	Critical	Architecture
43	One include block per data type	Cannot have two `logs` includes; combine conditions with `OR` within a single include	Critical	Architecture
44	Use Primary Grail Fields in segment filters	Filter on `dt.host_group.id`, `k8s.namespace.name`, `k8s.cluster.name`, `aws.account.id`, `azure.subscription`, `gcp.project.id` — these are indexed and propagate across all signal types	Critical	Performance
45	Use prefix-based naming for cross-signal filtering	Name host groups, namespaces, etc. with prefixes (e.g., `prod-web-tier` not `web-tier-prod`) so `starts-with` works on both entity and signal includes	Recommended	Naming
46	Create one segment per dimension	Separate segments for platform, application, environment, and region — users combine them as needed	Recommended	Architecture
47	Add event includes for detected problem filtering	Define events include with `event.kind = "DAVIS_PROBLEM"` — entity includes alone do not filter problems	Critical	Troubleshooting
48	Set official segments to Public visibility	Platform team-owned segments: set to Public; application team segments: share with their group	Recommended	Governance
49	Use consistent segment naming prefixes	Prefixes: `team-`, `env-`, `app-`, `region-` (e.g., `team-platform-infra`, `env-production`)	Recommended	Naming
50	Test segment filter conditions with DQL before creating the segment	Run the equivalent `filter` clause as a DQL query; verify record counts match expectations	Critical	Operations

#	Best Practice	Recommended Setting/Value	Priority	Category
51	Limit segment variable dropdowns to 10,000 values	Maximum 10,000 values per variable; 100 selected values per segment	Recommended	Architecture
52	Use entity relationship traversal for comprehensive segments	Top-down: host → process group → service; bottom-up: service → process group → host	Recommended	Architecture
53	Segments reduce scanned bytes	Applying a segment injects filters at query time, reducing data scanned and improving performance on high-volume buckets	Recommended	Performance

Validation Queries — Segment Design

Simulate segment filter conditions before committing to a segment definition:

```
// Simulate host-group-based segment filter on logs
fetch logs, from:-1h
| filter isNotNull(host.group)
| summarize count = count(), by:{host.group}
| sort count desc
| limit 10
```

```
// Simulate tag-based segment filter on services
fetch dt.entity.service
| expand tags
| filter contains(tags, "team:platform")
| fields entity.name, tags
| limit 20

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | expand tags
//   | filter contains(tags, "team:platform")
//   | fields name, tags
//   | limit 20
// Caveat: Smartscape reflects CURRENT live topology and can report fewer entities than
// the classic entity store; for a pre-migration inventory keep the classic query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via getNodeField).
```

```
// Simulate combined segment filter — verify host group + namespace combination returns
expected logs
fetch logs, from:-1h
| filter isNotNull(dt.host_group.id) and isNotNull(k8s.namespace.name)
| summarize count = count(), by:{dt.host_group.id, k8s.namespace.name}
| sort count desc
| limit 20
```

7. Metadata Enrichment

#	Best Practice	Recommended Setting/Value	Priority	Category
54	Enrich all signals with Primary Grail Fields before building segments	Verify `dt.host_group.id`, `k8s.namespace.name`, `k8s.cluster.name` are present on logs, spans, and metrics	Critical	Architecture
55	Set `dt.security_context` via host properties for dedicated infrastructure	Deployment Status → select hosts → Modify host properties → set `dt.security_context=<team>`	Recommended	Security
56	Set `dt.cost.costcenter` and `dt.cost.product` on hosts	Host properties: `dt.cost.costcenter=<center>`, `dt.cost.product=<product>` — these propagate to service metrics	Recommended	Cost
57	Use `DT_TAGS` environment variable for shared infrastructure	Set `DT_TAGS=primary_tags.team=<team>` per process when multiple apps share the same host	Recommended	Architecture
58	Restart applications after enrichment changes for spans	Logs: no restart needed (OneAgent auto-enriches); Spans: application restart required; Metrics: OneAgent restart applies automatically	Critical	Operations
59	Verify enrichment coverage before building segments	Run DQL audit query to check field coverage percentage across signal types; fields with 0% coverage need enrichment	Critical	Operations
60	Use Primary Grail Tags (`primary_tags.*`) when standard fields do not match your organization	Set via host properties or `DT_TAGS` environment variable; propagates across logs, spans, and topology	Recommended	Architecture

Validation Queries — Metadata Enrichment

Audit primary Grail field coverage before building segment definitions:

```
// Audit primary Grail field coverage on logs — low-coverage fields need enrichment before
segment use
fetch logs, from:-1h
| summarize
    total = count(),
    with_namespace = countIf(isNotNull(k8s.namespace.name)),
    with_cluster = countIf(isNotNull(k8s.cluster.name)),
    with_host_group = countIf(isNotNull(dt.host_group.id)),
    with_security_ctx = countIf(isNotNull(dt.security_context))
| fieldsAdd
    ns_pct = round(toDouble(with_namespace) / toDouble(total) * 100, decimals: 1),
    cluster_pct = round(toDouble(with_cluster) / toDouble(total) * 100, decimals: 1),
    hg_pct = round(toDouble(with_host_group) / toDouble(total) * 100, decimals: 1),
    sec_pct = round(toDouble(with_security_ctx) / toDouble(total) * 100, decimals: 1)
```

```
// Audit primary Grail field coverage on spans — spans require app restart to pick up new
enrichment
fetch spans, from:-1h
| summarize
    total = count(),
    with_namespace = countIf(isNotNull(k8s.namespace.name)),
    with_cluster = countIf(isNotNull(k8s.cluster.name)),
    with_host_group = countIf(isNotNull(dt.host_group.id)),
    with_service = countIf(isNotNull(dt.entity.service))
| fieldsAdd
    ns_pct = round(toDouble(with_namespace) / toDouble(total) * 100, decimals: 1),
    cluster_pct = round(toDouble(with_cluster) / toDouble(total) * 100, decimals: 1),
    hg_pct = round(toDouble(with_host_group) / toDouble(total) * 100, decimals: 1),
    svc_pct = round(toDouble(with_service) / toDouble(total) * 100, decimals: 1)
```

```
// Variable DQL: list host groups — use as segment variable for host-group-scoped filters
fetch dt.entity.host_group
| fields host_group = entity.name, id
| sort host_group asc

// Note: dt.entity.* is deprecated. There is no Smartscape "HOST_GROUP" node type; host-
group
// membership is exposed as the dt.host_group.id field on HOST nodes. Keep the classic
query above.
```

```
// Variable DQL: list Kubernetes namespaces — use as segment variable for namespace-scoped
filters
fetch dt.entity.cloud_application_namespace
| fields namespace = entity.name
| fieldsAdd tag = concat("Namespace:", namespace)
| sort namespace asc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_NAMESPACE"
//   | fields namespace = name
//   | fieldsAdd tag = concat("Namespace:", namespace)
//   | sort namespace asc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer entities than
// the classic entity store; for a pre-migration inventory keep the classic query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via getNodeField).
```

8. Query Performance

#	Best Practice	Recommended Setting/Value	Priority	Category
61	Understand the 500 GB scan limit	Maximum 500 GB scanned per query; at 1 TB/day ingest, queryable window is ~12 hours	Critical	Performance
62	Target specific buckets in queries	Use `fetch logs, bucket: {"team_logs", "app_logs_*"}` to avoid scanning all buckets	Recommended	Performance
63	Use exact match (`=`) over pattern match in segment filters	`=` is faster than `starts-with` ; use exact values when known	Recommended	Performance
64	Minimize segment filter expressions	Fewer conditions = better query performance; maximum 10 expressions per filter	Recommended	Performance
65	Use OpenPipeline pre-processing for complex segment logic	If segment conditions would be complex, pre-enrich a simple field (e.g., `dt.security_context`) and filter on that	Recommended	Performance
66	Maximum 1,000 records returned per query	Use aggregations (`summarize` , `makeTimeseries`) for larger datasets	Recommended	Performance
67	Maximum 10 segments per query	Do not stack more than 10 segments; design broader segments instead of many narrow ones	Recommended	Architecture

Validation Queries — Query Performance

Use bucket targeting to control scan scope and stay within the 500 GB scan limit:

```
// Query from multiple buckets simultaneously — reduce scan scope by naming targets
explicitly
fetch logs, from:-1h, bucket:{"default_logs", "audit_logs", "security_logs"}
| limit 100
```

```
// Filter by bucket within query — useful when bucket list is dynamic
fetch logs, from:-1h
| filter dt.system.bucket == "default_logs"
| filter loglevel == "ERROR"
| limit 50
```

9. Governance and Operations

#	Best Practice	Recommended Setting/Value	Priority	Category
68	Schedule regular access reviews	Quarterly review of all IAM policies and group memberships; remove unnecessary permissions	Critical	Governance
69	Document all bucket purposes	Maintain a registry mapping each bucket to its owner, data type, retention, cost center, and routing rule	Recommended	Governance
70	Document all security context values	Maintain a registry of all `dt.security_context` values, their meaning, and which IAM policies reference them	Recommended	Governance
71	Platform team owns infrastructure segments	Platform/SRE team creates and maintains public environment and infrastructure segments	Recommended	Governance
72	Application teams own app-specific segments	Each team creates segments for their applications; share with their IAM group	Recommended	Governance
73	Review segments quarterly	Remove unused segments; verify filter conditions still match data; check for empty results	Recommended	Operations
74	Use combined approach for defense in depth	Buckets for retention/cost + Security context for access control + Segments for dynamic views — all three together	Critical	Architecture

#	Best Practice	Recommended Setting/Value	Priority	Category
75	Implement in phases	Phase 1: Plan → Phase 2: Buckets + routing → Phase 3: IAM policies → Phase 4: Segments → Phase 5: Governance	Recommended	Operations
76	Selective record deletion is not possible	Cannot delete individual records; wait for bucket retention to expire, or truncate the entire bucket	Critical	Operations
77	Bucket deletion destroys all data permanently	Deleting a bucket removes ALL records immediately and irreversibly	Critical	Operations

Validation Queries — Governance and Operations

Routine audit queries for operational health checks:

```
// Audit bucket data distribution — verify all expected buckets are receiving data
fetch logs, from:-1h
| summarize
    recordCount = count(),
    by:{dt.system.bucket}
| sort recordCount desc
```

```
// Audit bucket retention settings — verify retention tiers match compliance requirements
fetch dt.system.buckets
| fields name, dt.system.table, retention_days
| sort dt.system.table asc, retention_days desc
```

Audit bucket management events — who created, updated, truncated, or deleted a bucket and when:

```
// Audit all bucket management actions — create, update, truncate, delete
fetch dt.system.events
| filter event.kind == "AUDIT_EVENT" and event.category == "BUCKET_MANAGEMENT"
| sort timestamp desc
```
